QuillAudits

AUDIT REPORT

August 2026

For

BITSTOCK

Table of Content

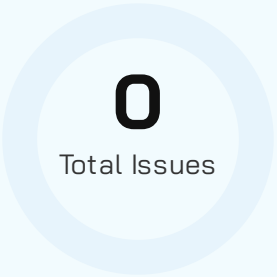
Executive Summary

Project Name	TBC Token
Protocol Type	ERC 20
Project URL	https://www.tenbestcoins.com/
Overview	The TBCToken is an ERC-20 token for TEN BEST COINS (TBC), with the full initial supply minted to the designated owner at deployment. It implements ERC20Permit (EIP-2612), enabling gas-efficient approvals through off-chain signatures. The contract includes an owner-controlled blacklist mechanism that freezes blacklisted addresses from sending or receiving tokens without destroying their funds. A low-privilege guardian can immediately blacklist suspected malicious addresses but has no authority to remove blacklists, manage ownership, or transfer tokens. Token distribution is supported through a gas-efficient batchTransfer function, with a maximum batch size of 300 recipients.
Audit Scope	The scope of this Audit was to analyze the TBC Token Smart Contracts for quality, security, and correctness.
Source Code link	https://drive.google.com/drive/folders/1_NV4SunzeswoIr_hUYQvYbICqu3vR3zP
Contracts in Scope	TBCToken.sol
Branch	NA
Commit Hash	NA
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	27th August 2026
Updated Code Received	NA

Review 2	NA
Fixed In	NA

✅ Verify the Authenticity of Report on QuillAudits Leaderboard:
https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	0 (0%)
Low	0 (0%)
Informational	0 (0%)

Severity

Issues

	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	0	0

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls
- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall

- Upgradeable safety
- Style guide violation
- Using throw
- Unsafe type inference
- Using inline assembly
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Functional Tests

Deployment & Supply

- Should mint the entire supply to owner_ and not to the deployer.
- Should correctly scale initialSupply_ using the token decimals.
- Should set owner_ as the contract owner.
- Should revert deployment when owner_ is the zero address.

Transfers

- Should allow the owner to transfer tokens to a valid recipient.
- Should allow any holder to transfer tokens from their own balance.
- Should revert transfers when the sender is blacklisted.
- Should revert transfers when the recipient is blacklisted.
- Should allow transfers normally after an address is removed from the blacklist.

Batch Transfer

- Should transfer the specified amounts to all recipients through batchTransfer().
- Should revert batchTransfer() when recipient and amount array lengths differ.
- Should allow a batch containing exactly 300 recipients.
- Should revert batchTransfer() when the number of recipients exceeds 300.
- Should revert when the caller does not have sufficient balance for the batch transfers.

Blacklist Management

- Should allow the owner to blacklist an address.
- Should prevent the owner from blacklisting the zero address.
- Should prevent the owner from blacklisting itself.

- Should prevent an already blacklisted address from being blacklisted again.
- Should allow the owner to remove an address from the blacklist.
- Should revert when removing an address that is not blacklisted.
- Should emit the appropriate AddedBlackList and RemovedBlackList events.

Allowance / transferFrom

- Should allow a holder to approve a spender and transfer tokens through transferFrom().
- Should prevent a blacklisted spender from using transferFrom().
- Should allow a spender to use transferFrom() after being removed from the blacklist.

Guardian

- Should allow the owner to set a guardian address.
- Should emit GuardianUpdated when the guardian is changed.
- Should allow the owner to replace or disable the guardian.
- Should allow only the guardian to call freezeAsGuardian().
- Should allow the guardian to blacklist a valid address.
- Should prevent the guardian from blacklisting the owner.
- Should prevent the guardian from blacklisting the zero address.
- Should prevent the guardian from removing an address from the blacklist.
- Should prevent the guardian from transferring or moving tokens.
- Should prevent freezeAsGuardian() from being used when no guardian is configured.

Ownership

- Should allow the owner to transfer ownership to a non-blacklisted address.
- Should prevent ownership from being transferred to a blacklisted address.
- Should prevent renounceOwnership() from succeeding.
- Should ensure ownership remains unchanged after a failed renounceOwnership() attempt.

ERC20Permit

- Should allow a token holder to approve a spender through a valid EIP-2612 permit signature.
- Should correctly update the allowance after a successful permit().
- Should reject an invalid permit signature.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

Dependency	Type	How Used	Trust Assumption	Risks
ERC20 (OpenZeppelin v5)	Base Contract	Provides balances, allowances, transfers and the _update mechanism	_update remains the common balance-changing path	If a future modification bypasses _update, blacklist restrictions may not apply
ERC20Permit (OpenZeppelin v5)	Base Contract	Provides EIP-2612 signature-based approvals	Permit implementation correctly validates signatures and nonces	Compromised private keys can authorize allowances
Ownable (OpenZeppelin v5)	Base Contract	Provides ownership and onlyOwner access control	Owner is securely controlled, preferably by a multi-sig	Compromise or loss of owner key affects blacklist and guardian management
Context (OpenZeppelin v5)	Library	Provides _msgSender() used by inherited functions	Correctly identifies the transaction caller	No material risk identified
Solidity compiler 0.8.24	Toolchain	Provides checked arithmetic and contract compilation	Compiler behavior is trusted and version is pinned	No arithmetic overflow issue identified

Dependency	Type	How Used	Trust Assumption	Risks
Owner address	Off-chain actor	Controls blacklist management, ownership and guardian configuration	Owner is honest and securely managed	Compromised owner can blacklist arbitrary addresses and configure the guardian
Guardian address	Off-chain actor	Can immediately blacklist addresses through freezeAsGuardian()	Guardian key is securely controlled and used only for incident response	Compromised guardian can freeze arbitrary non-owner addresses

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

Contract Name	Function	Category
TBCToken.sol	`constructor()`	**Main invariant(s)** Entire supply is minted exactly once at deployment **Access level** Deployment only **What this function can do** Sets token metadata and mints the complete initial supply to owner_ **What this function cannot/should not do** Cannot mint again or mint to the deployer unless deployer is explicitly supplied as owner_
TBCToken.sol	`batchTransfer(address[], uint256[])`	**Main invariant(s)** Arrays must have equal length; every transfer uses transfer() **Access level** Public **What this function can do** Transfers tokens from the caller to up to 300 recipients **What this function cannot/should not do** Cannot transfer tokens belonging to another address or exceed the batch limit

Contract Name	Function	Category
TBCToken.sol	`addBlackList(address)`	**Main invariant(s)** Blacklisted address cannot participate in transfers **Access level** onlyOwner **What this function can do** Blacklists an address, preventing it from sending or receiving tokens **What this function cannot/should not do** Cannot blacklist zero address or current owner; cannot blacklist twice
TBCToken.sol	`removeBlackList(address)`	**Main invariant(s)** Address becomes transferable again after removal **Access level** onlyOwner **What this function can do** Removes an address from the blacklist **What this function cannot/should not do** Cannot remove an address that is not currently blacklisted
TBCToken.sol	`setGuardian(address)`	**Main invariant(s)** Only the current owner can configure the guardian **Access level** onlyOwner **What this function can do** Sets, replaces, or disables the guardian

Contract Name	Function	Category
TBCToken.sol	`freezeAsGuardian(address)`	**Main invariant(s)** Guardian can only add blacklist entries **Access level** Guardian **What this function can do** Immediately blacklists a target address **What this function cannot/should not do** Cannot blacklist zero address or owner; cannot remove blacklists or move tokens
TBCToken.sol	`transfer(address,uint256)`	**Main invariant(s)** _update() enforces blacklist checks **Access level** Public **What this function can do** Transfers tokens from caller to recipient **What this function cannot/should not do** Cannot transfer when sender or recipient is blacklisted or balance is insufficient
TBCToken.sol	`approve(address,uint256)`	**Main invariant(s)** Only modifies allowance state **Access level** Public **What this function can do** Creates or updates an allowance for a spender **What this function cannot/should not do**

Contract Name	Function	Category
TBCToken.sol	`transferFrom(address,address,uint256)`	**Main invariant(s)** Allowance is reduced by transferred amount **Access level** Public + allowance **What this function can do** Transfers tokens from an approved holder to a recipient **What this function cannot/should not do** Cannot transfer if from or to is blacklisted; cannot exceed allowance
TBCToken.sol	`transferOwnership(address)`	**Main invariant(s)** New owner must not be blacklisted **Access level** onlyOwner **What this function can do** Transfers all ownership privileges to a new owner **What this function cannot/should not do** Cannot transfer ownership to a blacklisted address
TBCToken.sol	`renounceOwnership()`	**Main invariant(s)** Ownership must remain available for blacklist recovery **Access level** onlyOwner **What this function can do** No state change; always reverts **What this function cannot/should not do**

Contract Name	Function	Category
TBCToken.sol	`permit(...)`	**Main invariant(s)** Valid signature required for allowance authorization **Access level** Public **What this function can do** Sets allowance through a valid EIP-2612 signature **What this function cannot/should not do** Cannot bypass signature, nonce, or deadline validation
TBCToken.sol	`View functions`	**Main invariant(s)** Read-only, no state changes **Access level** Read-only access **What this function can do** Expose token balances, allowances, ownership, blacklist status, supply and configuration **What this function cannot/should not do** Cannot modify state

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

Contract	Asset Type	Custodied Assets
TBCToken.sol	TBC ERC-20	No intended custody. Tokens accidentally sent directly to the contract cannot be recovered because there is no rescue/sweep function
Owner / Treasury	TBC ERC-20	Entire initial supply at deployment and any undistributed tokens remaining afterward
Token Holders	TBC ERC-20	Distributed token balances

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TBCToken.sol	constructor()	—	Full initial supply to owner_	Deployer	Incorrect owner_ permanently places the initial supply and administrative authority under the wrong address
TBCToken.sol	transfer()	Caller balance	Recipient balance	Public	Blocked when either party is blacklisted
TBCToken.sol	transferFrom()	Approved holder balance	Recipient balance	Public + allowance	Blacklisted spender is explicitly prevented from using another holder's allowance

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TBCToken.sol	batchTransfer()	Caller balance	Up to 300 recipients	Public	Entire transaction reverts if any individual transfer fails
TBCToken.sol	addBlackList()	—	—	Owner	Freezes the target's existing balance; does not move or destroy tokens
TBCToken.sol	removeBlackList()	—	—	Owner	Restores transferability to the target
TBCToken.sol	freezeAsGuardian()	—	—	Guardian	Emergency freeze capability; guardian cannot move funds
TBCToken.sol	approve()	—	Allowance state	Token holder	Does not itself move tokens
TBCToken.sol	permit()	Valid signature	Allowance state	Any relayer	Authorization depends on holder's private key
TBCToken.sol	(no function)	Tokens sent directly to contract	—	Anyone	Tokens become permanently inaccessible because no recovery function exists

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of TBC Token. We performed our audit according to the procedure described above.

No Issues Found in TBC Token Contract.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

August 2026

For

BITSTOCK

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com